*Chapter #7*

# *Unlocking the Secrets of Bitcoin's Inner Workings: The Math, Mempool, and UTXOs*

**7.0** Putting the Double Spend Issue to Rest: Understanding Bitcoin's Solution
**7.1** Tracking Your Coin's Journey
**7.2** Security and Secrecy
**7.3** The "Mempool" or Memory Pool: Understanding the Holding Tank of Bitcoin Transactions
- **7.3.1** Class Exercise: On Hold: Examining the Unconfirmed Transactions of the Bitcoin Network

**7.4** Behind the Blocks: The Mystery of Bitcoin Scripting
- **7.4.1** A Technical Dive into Bitcoin Transactions

**But how does Bitcoin actually work?** Watch the following video to understand better.

So far, we've learned about the history of money and the revolutionary idea of *blockchain* technology, and we've explored the basics of ***Bitcoin*** - the world's first decentralized digital currency. But how does ***Bitcoin*** prevent fraud and ensure that people can't spend the same money twice? That's where the magic of UTXOs, public key cryptography, hashing, scripting, and the mempool come into play. Just like a fingerprint ensures that nobody else can use your identity, hashes in ***Bitcoin*** ensure that transactions can't be altered. Scripts are like the rules for a game, making sure transactions follow specific conditions. UTXOs are like the building blocks of a puzzle, keeping track of all the money in your virtual wallet. And the mempool acts like a holding area, making sure all transactions are verified before being added to the *blockchain*. So, let's dive in and discover how ***Bitcoin*** solves the double-spending problem and ensures the integrity of every transaction on its network.

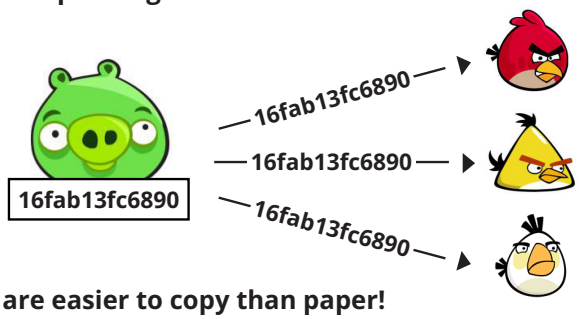
## *7.0 Putting the Double-Spend Issue to Rest: Understanding Bitcoin's Solution*

***Remind me, what is the "double spending problem"?***

***Bitcoin*** transactions, as we've experienced, are represented by a series of random numbers and letters that can be viewed on any device with internet access.

Additionally, much of the information related to these transactions is typically communicated using a numerical notation system known as **hexadecimal numbers**.

**Double spending ...**




**Bits are easier to copy than paper!**

This means that it is common to see strings of 64-character hexadecimal numbers consisting of letters (A-F) and numbers (0-9), such as

f4184fc596403b9d638783cf57adfe4c75c605f6356fbc91338530e9831e9e16

So how do we actually prevent someone from copying and pasting their bitcoin and spending it multiple times, like they would with an email or photo?

f4184fc596403b9d638783cf57adfe4c75c605f6356fbc91338530e9831e9e16.
f4184fc596403b9d638783cf57adfe4c75c605f6356fbc91338530e9831e9e16.

How do we reach consensus on who owns what money without a central authority?

**Hashing** and code. Let's explain.

Imagine you have 1 ***bitcoin*** and you want to send it to your friend as a birthday gift. You send the ***bitcoin*** to your friend's address, but then you realize that you owe your ex-boyfriend money and should have sent the ***bitcoin*** to him instead. In a moment of panic, you decide to be sneaky and create a new transaction to send the same 1 ***bitcoin*** to your ex-boyfriend. This is what we call a "double spend."

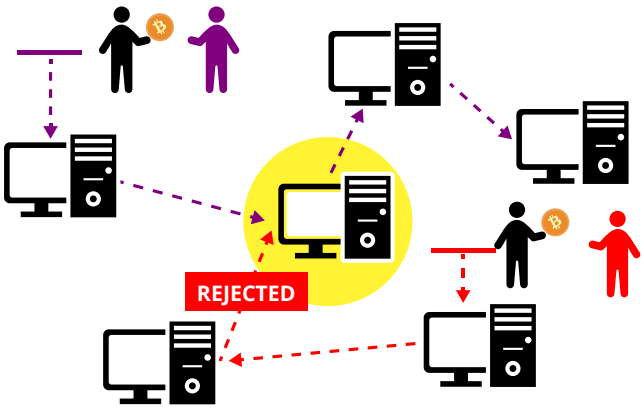
But wait, how does the network prevent this from happening? It's simple. The nodes on the network detect conflicting transactions and only allow one to go through based on a set of rules known as the "**consensus rules**." In this case, it's likely that the transaction to your ex-boyfriend would be rejected since it was sent after the original transaction to your friend. However, it's just a matter of luck which transaction is picked up first by a miner.

Thanks to the *blockchain*, **everyone on the network is able to agree on the current state of the ledger**. This helps to prevent **double-spending** and fraud, making it a secure and trustworthy system, just like a digital version of the "honor system." So, next time you send ***bitcoin***, you can relax knowing that the network has got your back.

So how does ***Bitcoin*** actually solve this? Well, let's find out.

- ***Bitcoin*** prevents double-spending by implementing a confirmation mechanism and maintaining a universal ledger (*blockchain*).
- Transactions are added to the *blockchain* in a chronologically-ordered, time-stamped manner.
- To prevent double spending, only the first transaction to receive enough confirmations (usually 6) is included in the *blockchain*, while the others are discarded.
- Transactions on the *blockchain* are irreversible and impossible to tamper with.

**Bitcoin Mining Node Rejects Transaction from Mempool**



When any transaction is initiated, any **node** can verify it on the network in a few simple steps:

**1.** First, the node will **check that the transaction is properly signed by the sender's private key**, which ensures that the transaction is legitimate and not tampered with.

**2.** Next, the node will **check that the sender has enough *funds*** to complete the transaction. This is done by examining the sender's balance on the blockchain ledger.

**3.** Finally, the node will also validate the transaction **inputs** and **outputs**, making sure that the inputs being spent in the transaction are not already spent in another transaction and that the outputs are not exceeding the total supply.

As we'll see, the combination of **public key cryptography** and the **UTXO** (Unspent Transaction Output) system is used in **Bitcoin** to verify the authenticity of transactions and prevent fraud without a central authority. **Public key cryptography** ensures secure communication and transfers of funds, while UTXO maintains a record of all the funds on the network and prevents double-spending.

**UTXO**, which stands for "Unspent Transaction Output," is simply a record of all the available funds in the network that have not yet been spent.

## *7.1 Tracking Your Coin's Journey*

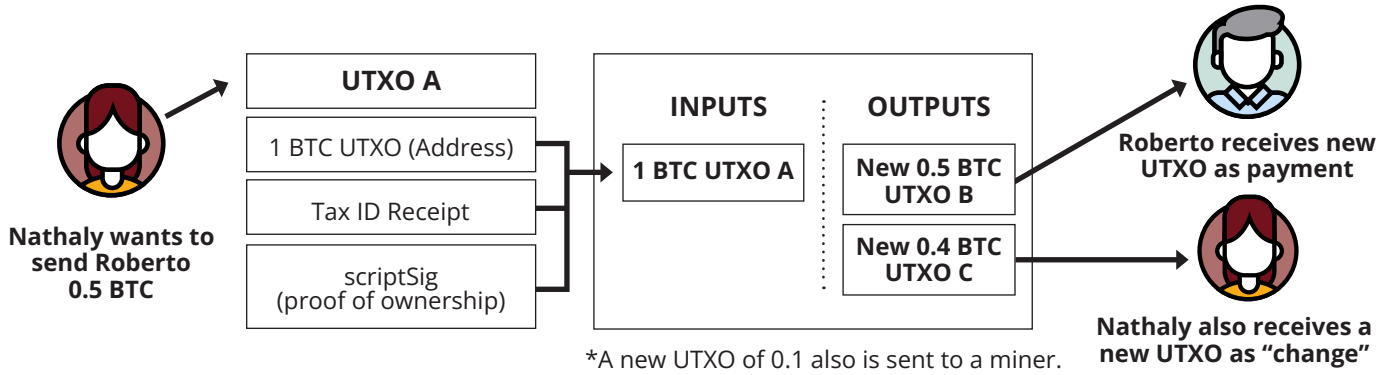
In ***Bitcoin***, transactions work like breaking a big bill into smaller bills and giving them to different people. The change you receive from a transaction is called an unspent output and can be used as an input for a new transaction. Outputs in ***Bitcoin*** transactions can either be **spent** or **unspent**, and unspent outputs are considered valuable because they can be used in new transactions.

- Think of it like using multiple gift cards to pay for a purchase. The gift cards from previous transactions act as inputs, and the change you receive is represented by a new gift card with the remaining amount. This is similar to how ***Bitcoin*** transactions work with UTXOs.

### *What are UTXOs?*

The balance of a wallet is the sum of all a user's UTXOs. UTXOs are used to track the ownership of ***bitcoin*** in the network. When a transaction is made, it creates new UTXOs, and when a transaction is spent, it uses up existing UTXOs.

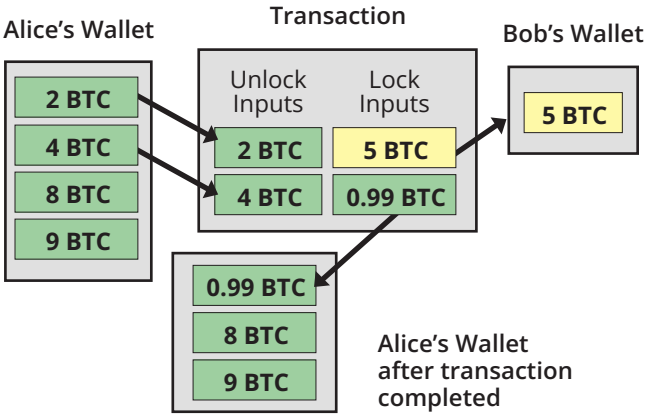
- UTXOs are like digital coins in the world of ***Bitcoin***. It is the change that you get back after you spend some ***bitcoin***.




***How UTXOs Work in Bitcoin Transactions***

When a transaction is made, the amount of ***bitcoin*** that is sent is divided into multiple outputs, each of which is associated with a specific address.

- When sending ***bitcoin*** to someone, you will use one or more Unspent Transaction Outputs (UTXOs) as the source of the funds. These UTXOs will be combined, if necessary, to create a new output that belongs to the recipient of the transaction. This new output, or UTXO, will then become the recipient's property and can be used as the source of funds in a future transaction. This chain of UTXOs creates a transparent and traceable history of all ***bitcoin*** transactions on the *blockchain*, starting from the very first block.




- For example, if someone wants to send 2 ***bitcoin*** but has a UTXO worth 5 ***bitcoin***, the difference of 3 ***bitcoin*** is sent back to the sender as "change." This change is a new UTXO for the sender and can be spent in a future transaction.

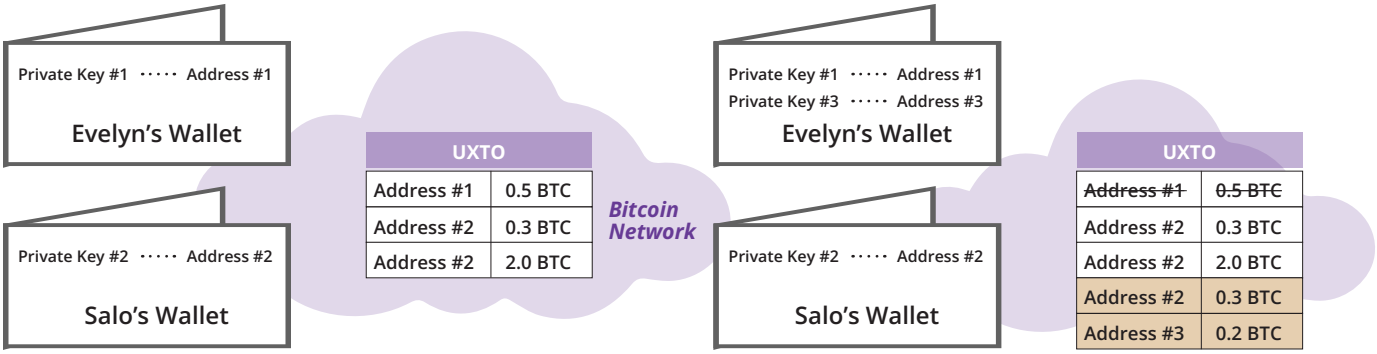
- In the example, Alice sends Bob 5 BTC while retaining some for herself. She combines 6 ***bitcoin*** from her four UTXOs, which totals 23 ***bitcoin***, and sends 5 to Bob and .99 back to herself, with a .01 fee for processing. The transaction is then added to the *blockchain*, updating all nodes with a copy of the updated UTXO ledger. If Alice then tries to send 23 BTC to Ximena in a separate transaction, the nodes will reject it as some of the output has already been spent.

If someone were to attempt to use a spent output in their transaction, it would likely be rejected by the network nodes. This is because these nodes maintain a copy of the same database set and can easily reach a consensus by checking the balance of each address before validating any new transaction. This ensures the integrity and validity of the transactions on the network.

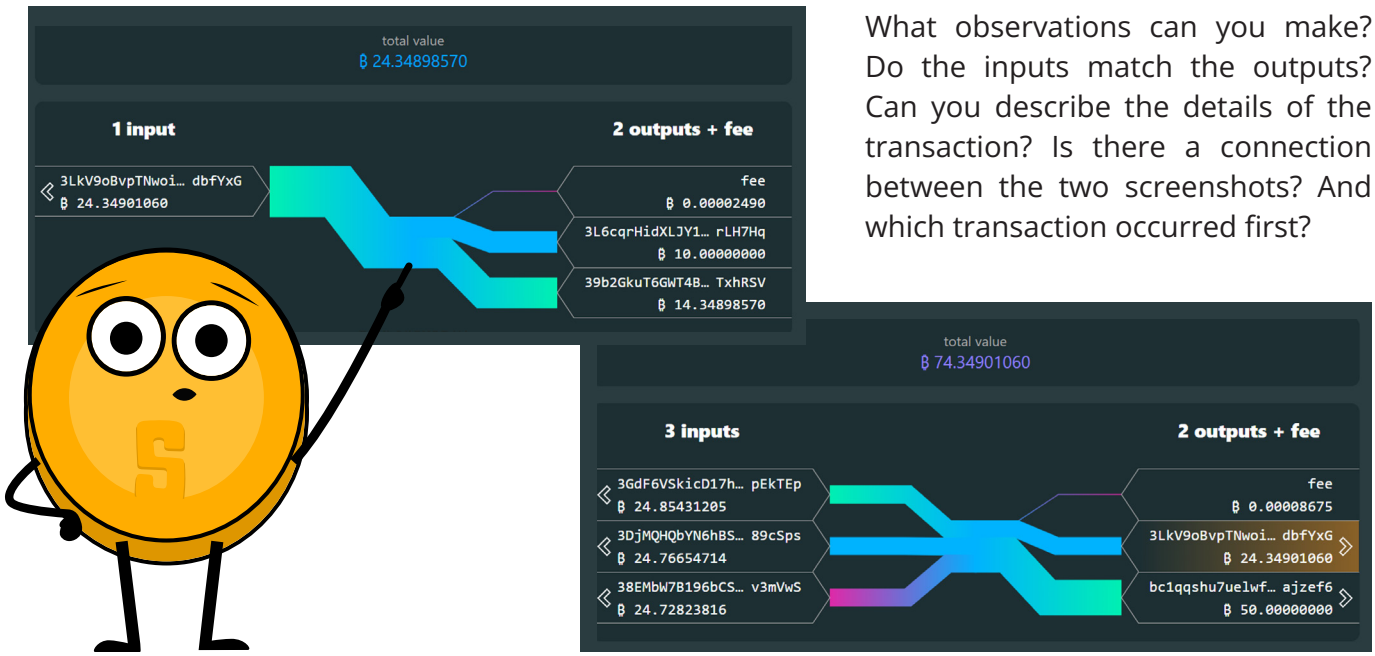
Let's look at another example:

Only the person who has the **private key** for an **address** can access the UTXOs stored in that address. For example, if Evelyn has a **private key** for **address** #1, she will see 0.5 ***bitcoin*** in her wallet. If Salo has a **private key** for **address** #2, he will see 2.3 ***bitcoin*** in his wallet.

When Evelyn sends 0.3 ***bitcoin*** to Salo, her wallet generates a new **private key** and address (#3). The original UTXO on **address** #1 gets spent and two new UTXOs are created: one for Salo's address with 0.3 ***bitcoin*** and one for Evelyn's new **address** with 0.2 ***bitcoin***. After this transaction is recorded in the ledger, Salo's wallet shows 2.6 ***bitcoin*** and Evelyn's wallet shows 0.2 ***bitcoin***.




Below is an actual screenshot of a real transaction where there is only one input. However, in a more general case, the starting balance could be the sum of multiple UTXOs that a person has accumulated from previous transactions.




What observations can you make? Do the inputs match the outputs? Can you describe the details of the transaction? Is there a connection between the two screenshots? And which transaction occurred first?

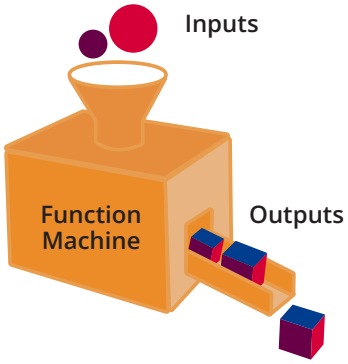
Generally, **spent outputs** are displayed in red and **unspent outputs** in green. This color coding provides a visual way to quickly identify spent and unspent outputs, which can be useful for tracking transactions and understanding the flow of funds in a blockchain.

## *7.2 Security and Secrecy*

Please don't be intimidated by the technical terms and mathematical concepts ahead. We understand that not everyone is a math junkie, but you might surprise yourself and see that even the most complex ideas can be grasped with a little bit of effort.

***What is a function, but more specifically, what is a one-way function?***

A **function** is like a machine that takes some information and turns it into something new. The information you give the function is called the **input**. The new information the function makes is called the **output**. Functions help computers do tasks and solve problems.




Think of it like a recipe for making a salad. The recipe (or function) tells you what ingredients to use and how to mix them together to make the salad. You can put different ingredients in, but the recipe will always give you the salad as the output. Functions can be used to help make things easier and more efficient.

This recipe therefore is a **function** that takes the **ingredients** as **inputs** and generates the **tossed salad** as the **output**.

# Unlocking the Secrets of Bitcoin's Inner Workings: The Math,

In ***Bitcoin***, **functions** are utilized to **execute transactions**. We already know that transactions in ***Bitcoin*** are essentially transfers of value from one address to another. To perform a transaction, a number of cryptographic functions are used to validate the transaction and update the state of the ***Bitcoin*** blockchain, which is a decentralized ledger that keeps track of all transactions.

The functions used in a ***Bitcoin*** transaction include verifying the authenticity of the transaction inputs, checking that the sender has sufficient funds, and updating the balances of the relevant addresses. Once a transaction is verified and added to the blockchain, it becomes part of the permanent record of all transactions on the network.

- A **one-way function** uses a set of instructions to process the information and turns it into something **new**, like a smoothie recipe turns ingredients into a new drink. But, just as **you can't un-blend a smoothie** to get the original ingredients back, **you can't reverse the one-way function to get the original information back.**

**Public-key cryptography**, of which **public key** is a part of, relies on the use of **one-way functions**, which make it difficult to determine the **private key** from the **public key**. It is not exactly "impossible" to find the private key from the **public key**, but it is extremely difficult to do so, and it would take an inordinate amount of time and computational power to accomplish this task.

- Finding a **private key** from a **public key** in ***Bitcoin*** is like trying to find a needle in a haystack as large as a football field. The needle represents the **private key** and the haystack represents all the possible **private keys**.

In the same way, one-way functions are designed to be irreversible and cannot be decrypted.

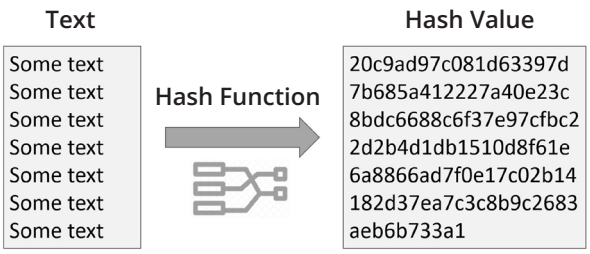
***What is a hash function?***

**Hashing** is like a fingerprint for digital data. It is a process of taking a digital message and turning it into a fixed length code, which serves as a unique identifier.

Just like a fingerprint can identify a person, a hash can identify a digital message. Hashes are used in many applications, including ***Bitcoin*** transactions.

### *How Hashing is Used in Bitcoin Transactions*

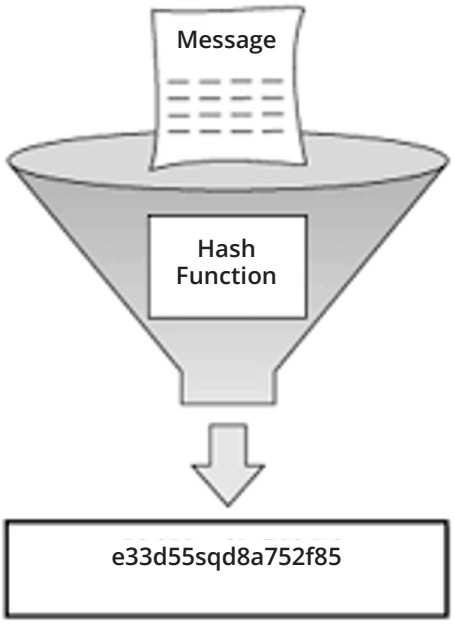
In ***Bitcoin***, every transaction is hashed before it is added to the *blockchain*. The hash acts as a signature for the transaction, verifying that the transaction is valid and has not been tampered with. If someone tries to change even a single letter in the transaction, the hash will be completely different, alerting others to the change.




### *The Role of Hashing in Providing Security*

Hashing is essential to the security of the ***Bitcoin Network***. By using hashes to identify transactions, the network can detect any attempt to change or manipulate a transaction. This helps to prevent fraud and ensure that all transactions are recorded accurately on the *blockchain*.

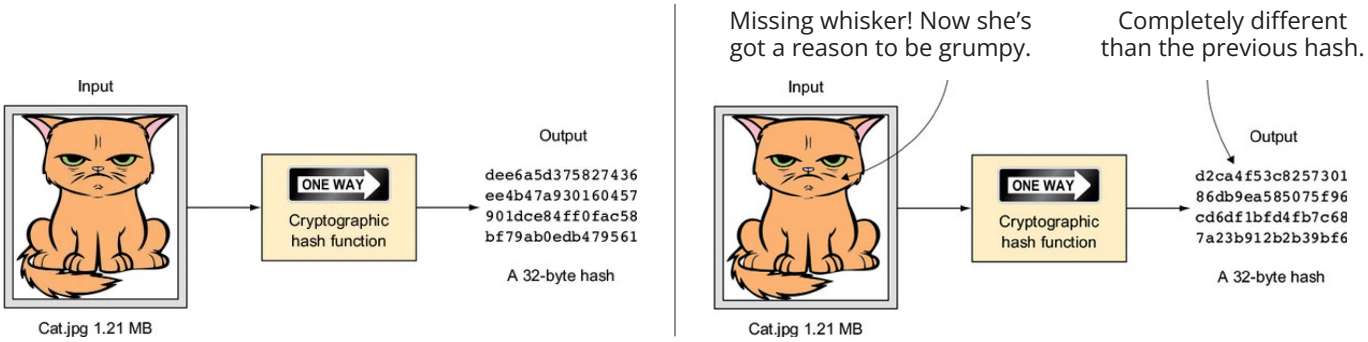
A hash function is a type of **one-way function** that takes an **input** (referred to as the "message" or "data") and converts it into a numerical representation referred to as a "**hash**." The **output** hash is unique to the **input** data, so even a small change in the input data results in a completely different hash.




A **hash function** is like a secret code machine. It takes in a **message** and turns it into a code.

- The code always looks the same for the same message. If you change the message even a little, the code will be completely different. This helps computers remember things and check if anything has been changed.

Instantly generate a SHA256 hash of any string or input value. Hash functions are used as one-way methods.




The **output**, or **hash**, is always the same length, no matter how long the original information was.

***Bitcoin*** uses a few specific types of hash **function** called **SHA-256 and RIPEMD160**.
A few examples below:

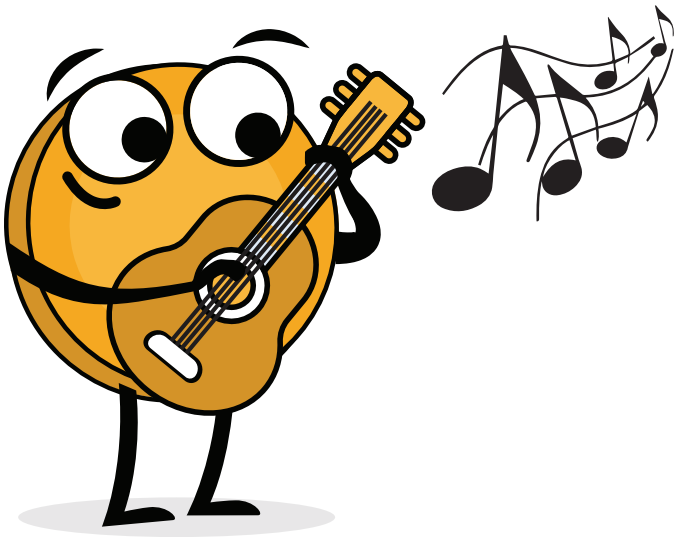
- Notice that a period in the second input changes the output completely when compared to the first one.
- The third input is a huge file yet the output is still the same fixed length as the other two.

SHA256 hash of the string ***hello world***
B94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9

SHA256 hash of the string **hello world.**
7ddb227315f423250fc67f3be69c544628dffe41752af91c50ae0a9c49faeb87

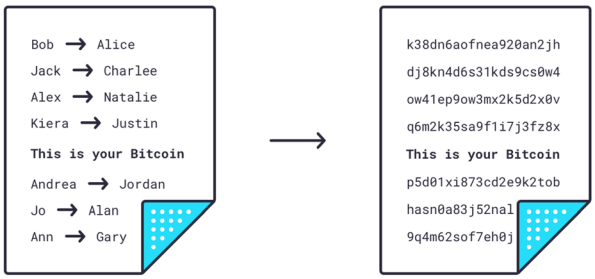
SHA256 hash of the downloadable iso file **Ubuntu 18.10**
7b9f670c749f797a0f7481d619ce8807edac052c97e1a0df3b130c95efae4765

**Hashing** can also be thought of as a musical score that captures the essence of a piece of music. Just as a musical score is a unique representation of a tune, a hash value is a unique representation of a piece of data. By comparing the score of a piece of music with the actual performance, a musician can determine if the performance is accurate. Similarly, by comparing the hash value of received data with the original hash value, one can determine if the data has been altered during transmission.



- Just as a slight deviation in a musical performance can cause it to sound different, even the slightest change to the original data will result in a different hash value. This makes hashing a powerful tool for ensuring the integrity and authenticity of digital information.

The process of encoding the **public key** through hashing is used to improve the security of information by converting it into a fixed-length, unreadable format. Bitcoin uses the SHA-256 and Ripemd-160 algorithms to produce public addresses. The resulting output serves as a unique identifier for the **public key** and helps to ensure the integrity and security of transactions stored in the *blockchain*. By **encoding** the information in this way, it becomes more difficult for unauthorized individuals to access and manipulate the data.

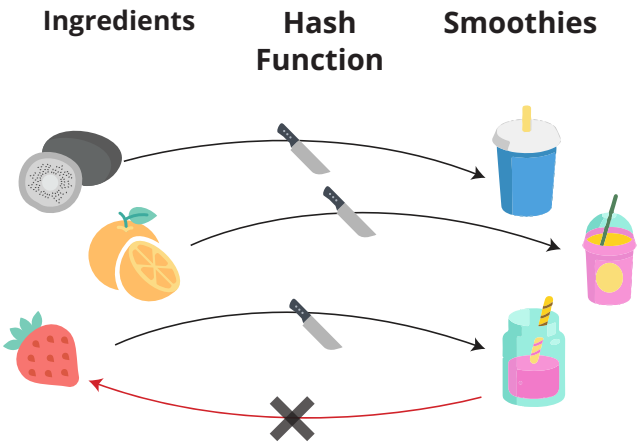


**In this link you'll find another hash generator where you can try it out yourself.**

---

**Hashing**

A hash function takes any input, and produces a fixed-length output (hash).




- ***Deterministic.*** The same ingredients always yield the same smoothie.

- ***Pre-Image Resistance.*** You can't glue together a strawberry when given a smoothie.

- ***Correlation Resistance.*** Changing the ingredients a little results in a completely different smoothie.

- ***Collision Resistance.*** It's hard to find different ingredients for a smoothie that result in the exact same one.

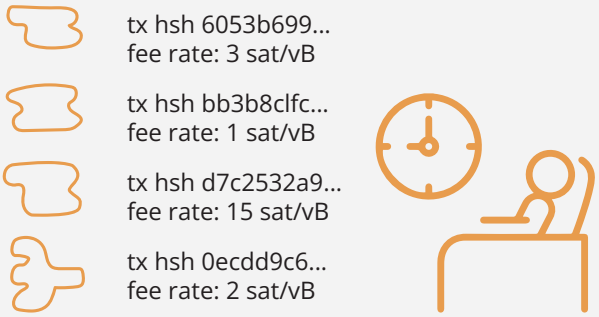
- ***Speed & Verifiablility.*** Throw fruit into the mixer. It's fast and what comes out for sure is a smoothie.

## *7.3 The "Mempool" or Memory Pool: Understanding the Holding Tank of Bitcoin Transactions*

### *What is the Mempool?*

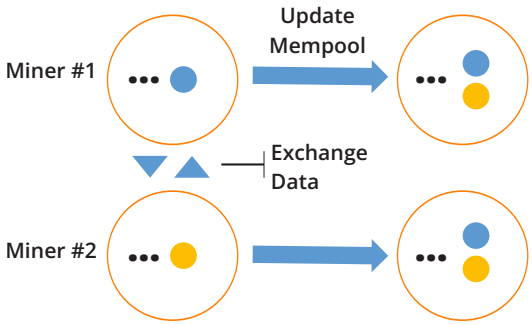
The mempool is like a waiting room for transactions in the ***Bitcoin Network***. When a transaction is made, it is first added to a node's mempool before it is verified and added to the *blockchain*.

A **mempool** is where transactions wait to be confirmed into a block.




When a node fist receives a transaction from a peer, it has to verify the transaction is legit. Nobody wants faulty or deceptive transactions.

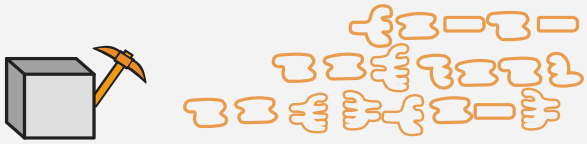
**Mempool synchronization** allows nodes to share their transactions with other nodes by sending a message containing a list of **verified** transactions in the mempool.




The main purpose of a **mempool** is to:

**1** Relay unconfirmed transactions.

**2** Provide miners transactions to mine.



**Accept To Memory Pool (ATMP)** involves checking things like:

- Do I already have this transaction?
- Is there a conflict with a different transaction in the mempool?
- Does the ***bitcoin*** in cover the ***bitcoin*** out?
- Do the signatures prove the previous outputs can be spent?
- Are there enough fees?

***How Transactions are Verified and Added to the Mempool***

A **full node** checks all transactions to make sure they are valid and have not been used before. If a transaction is good, the node will verify it and add it to it's memory pool. Then it will share it with other nodes to double-check. Finally, if the majority agree, it will be taken out of everyone's mempool to become permanent part of the *blockchain*.

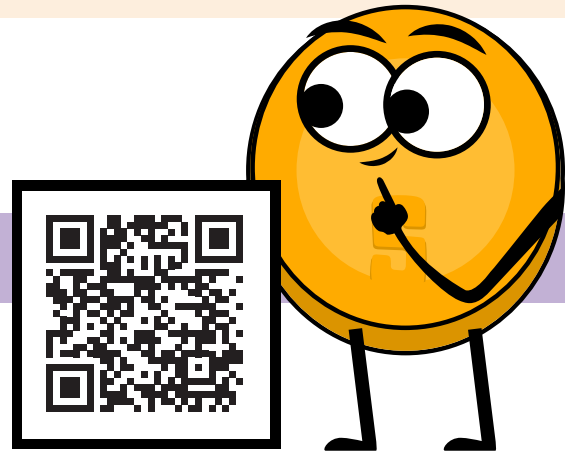
Transactions in the ***Bitcoin Network*** are taken out of the mempool and confirmed when they are included in a block, which is then added to the *blockchain*. However, there are several reasons why a transaction might not be confirmed after 72 hours:

**1. Low fee**: Transactions with a low fee may not be processed quickly enough, as miners are more likely to choose transactions with higher fees to include in their blocks.
**2. Network congestion**: If the network is congested, there may be a delay in confirming transactions, even if they have a high fee.
**3. Double spend attempt**: If a malicious actor attempts to double spend, their transaction may be rejected by the network.
**4. Incorrect or incomplete data**: If a transaction contains incorrect or incomplete data, it may be rejected by the network.
**5. Malformed transaction**: If a transaction is malformed, it may be rejected by the network.

To avoid having transactions rejected, it's recommended to include a fee that is high enough to ensure the transaction is processed in a timely manner, and to double check that all the data in the transaction is correct before sending it.

A **DDoS** (Distributed Denial of Service) attack is an attempt to make the network unavailable to users by overwhelming it with too much traffic from multiple sources. This attack aims to disrupt normal traffic of a website or service by overwhelming it with fake traffic, making it difficult or impossible for real users to access it.

### *7.3.1 Class Exercise: On Hold: Examining the Unconfirmed Transactions of the Bitcoin Network*



**Class Exercise.** Follow the following instructions:

**1.** Visit the website *https://bits.monospace.live/*

**2.** Locate an unconfirmed transaction and click on it.

- What information can you find?
- Can you follow the history of where bitcoin came from?
- How many addresses do you see? What does input and output mean?
- Can you follow the UTXO? Do you recognize which BTC has been spent?
- Does the input match the output?
- Does every transaction have a fee?
- Who does the fee goes to? Is it fair?
- Who pays the fee?
- Can you locate how much bitcoin was transferred from one address to another?

**3.** Write down the TxID, fee rate, fee, and total value of the transaction in a notebook.

**4.** Analyze other transactions if desired, and compare them to the first one in terms of amount, fee paid, and likelihood of being included in the next block.

**5.** Consider what it means for a block to be "mined" and for a transaction to be "unconfirmed."

**6.** Be prepared to discuss these observations and questions in the next class.

### *7.4 Behind the Blocks: The Mystery of Bitcoin Scripting*

**Script** is a **programming language** used in ***Bitcoin*** to create **smart contracts** and **automate** transactions. To understand script, it's helpful to think of it as a set of instructions that tell the ***Bitcoin Network*** what to do with a specific transaction.

A **smart contract** is a self-executing contract with the terms of the agreement between buyer and seller being directly written into lines of code. The code and the agreements contained within it exist on a blockchain network and are enforced automatically.

- Think of it like a vending machine. You put in money, make a selection, and the machine dispenses your item automatically. In the same way, a smart contract automatically executes the terms of the agreement between two parties, without the need for intermediaries such as lawyers or banks.

For example, a smart contract could be used to represent a financial agreement, such as a loan or a bond. The terms of the agreement, such as the interest rate and repayment schedule, are encoded into the contract. When the agreed-upon conditions are met, the contract automatically executes the terms and transfers the funds.

The key benefits of smart contracts are that they are transparent, secure, and self-executing, which can help to reduce the costs and risks associated with traditional contract processes. Additionally, because they exist on a decentralized network, they are resistant to tampering or interference, making them a more secure and trustworthy way of conducting transactions.

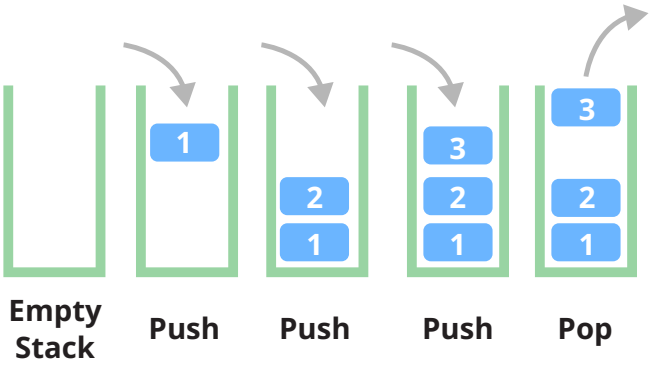
In a similar way, ***Bitcoin*** uses script to make sure that specific conditions are met before a transaction is processed.

While other *blockchain* networks, such as Ethereum, also support smart contracts and programmable transactions, they use different programming languages and approaches to enforce the rules and conditions of transactions. Only Bitcoin uses script.

Script is a very basic programming language, but it's powerful enough to handle a wide range of transactions. For example, it can be used to create **multi-signature** transactions, where multiple people must sign off on a transaction before it can be processed, or to create a smart contract, where a transaction is automatically executed when certain conditions are met.

While script may seem complex, the basic idea behind it is actually quite simple. By using script, the Bitcoin network can automatically enforce the rules and conditions of transactions, making it a secure and efficient way to transfer value.

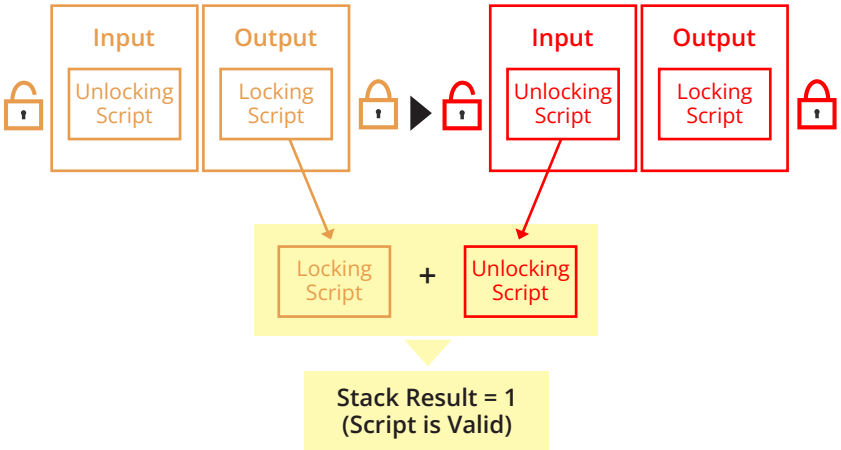
***How Scripting is Used in Bitcoin Transactions***




- Imagine you have a row of coins and you want to sort them into different slots, like putting coins into different piggy banks. The order in which you put the coins into the slots is important. This is similar to how satoshis are transferred in a transaction. The inputs are like a row of coins and the outputs are the slots waiting to receive a coin. To assign the coins to the slots, you go through each coin in the row in order, and put each one into the first available slot. This is called "first-in-first-out" or FIFO, which means that the first coin in the row goes into the first available slot, and so on.

- Scripts operate on a **stack-based system**, where instructions are processed in the order they appear, from top to bottom.The concept is similar to a stack of plates, where you can only access the plate that is on top of the stack.

A basic bitcoin transaction uses at least one "**locking script**" and one "**unlocking script**" to determine who can access the funds in a particular wallet address. The locking script can be thought of as a list of instructions that describes how the recipient of the funds can access them, while the unlocking script unlocks the funds.



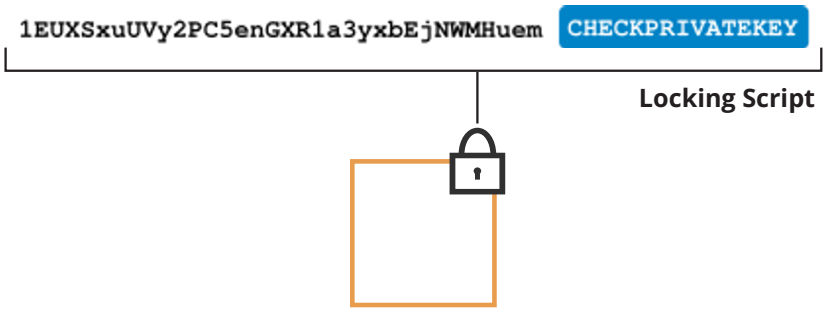

- Think of Script as a recipe for baking a cake. Just as you need to follow the steps in the recipe to make the cake, the computer needs to follow the instructions in Script in a specific order to transfer ownership of ***bitcoin***.

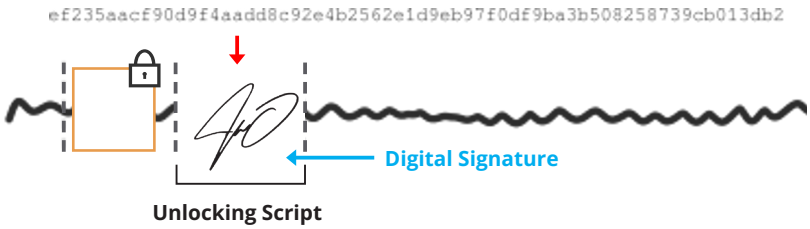
By using locking and unlocking scripts, along with private and public keys, the ownership and transfer of UTXOs can be securely tracked and verified.

### *7.4.1 A Technical Dive into Bitcoin Transactions*

The locking script holds the **recipient's address** and verifies that the correct **private key** was used. This ensures that **private keys** remain confidential and can be securely protected.

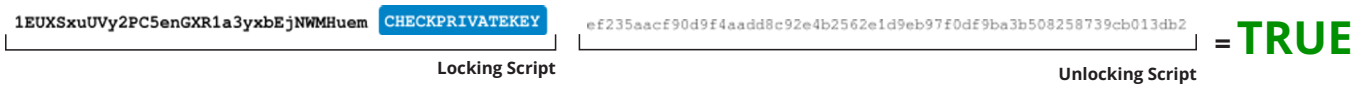
To unlock the funds, the sender must demonstrate ownership by generating a digital signature with their **private key**, thereby confirming possession of the **address**.

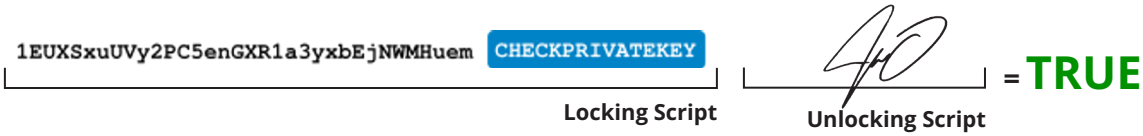





For example, let's say that you want to send some ***bitcoin*** to your friend, but you want to make sure that your friend can only spend them after a certain date. You can use ***Bitcoin*** script to define this condition, which is known as a "**time-lock**". When you create the transaction, you include a script that specifies the time-lock condition. When your friend receives the ***bitcoin***, they can only spend them after the specified date has passed.

***Bitcoin*** script can also be used to create more complex conditions for spending ***bitcoin***, such as multi-signature transactions, which require multiple parties to authenticate a transaction before it can be spent. This can be useful in situations where multiple parties need to approve a transaction.




In simple terms, script helps ensure the security and reliability of ***Bitcoin*** transactions by using private and public keys to verify ownership and transfer of funds. Different methods of transactions have varying levels of security. Some reveal the recipient's **public key** during the transaction, making it vulnerable to theft if the **private key** is ever hacked. Others keep the **public key** hidden, providing a higher level of security.

In the next chapter, we will delve deeper into the process of mining and the role of miners in the ***Bitcoin Network***. We will explore how they validate transactions, create new blocks, and receive rewards for their efforts.

Stay tuned for a comprehensive understanding of how the ***Bitcoin Network*** operates!